UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

POSGRADOS

Maestría en

SOFTWARE

Asignatura:

Patrones de Diseño de Software

Actividad #: 01

Patrones creacionales

Docente:

Mauricio Ortiz Ochoa

Elaborado por (Grupo 02):

Espinoza Adrian
Jara Pablo
Lara Miguel
Torres Carina
Uyaguari Sebastián

Fecha de entrega:

2026/09/17

Repositorio de código:

https://github.com/miguesj-hub/patrones-diseno-actividad-01

Introducción

El presente informe documenta la resolución de la Actividad de aprendizaje 01 de la asignatura Patrones de Diseño de Software, orientada al resultado de aprendizaje RA1: seleccionar y aplicar patrones creacionales para resolver problemas de creación de objetos, revisando críticamente las alternativas elaboradas con apoyo de inteligencia artificial.

Se abordan dos escenarios. En el primero, el registro de vehículos y el cálculo de su matrícula, se aplica el patrón **Factory Method**; en el segundo, el registro de activos fijos institucionales, se aplica el patrón **Builder**. Para cada escenario se presentan la identificación del problema de creación de objetos, la selección y justificación del patrón, sus participantes, el diagrama de clases UML, la implementación en Java con su correspondencia con el diagrama, y el análisis del uso crítico de inteligencia artificial. El informe concluye con las conclusiones del grupo y las referencias consultadas.

El código fuente de ambas soluciones se encuentra en el repositorio del grupo: https://github.com/miguesj-hub/patrones-diseno-actividad-01.

Escenario 01: Registro de vehículos y cálculo de matrícula (Factory Method)

Justificación

Identificación del problema de creación de objetos

La empresa requiere registrar vehículos y calcular el valor de su matrícula para tres tipos concretos: Auto, Camioneta y Camión. Todos comparten un conjunto de datos (placa, marca, modelo, año de fabricación y avalúo) y una operación común para obtener el costo de matrícula; sin embargo, cada tipo aplica una regla de cálculo distinta que puede depender de atributos propios, como la capacidad y el uso en la camioneta o el tonelaje en el camión.

El análisis del enunciado permite identificar que el problema central no es el cálculo en sí, sino **la creación del objeto**: el tipo concreto de vehículo se conoce únicamente *en tiempo de ejecución*, a partir de la información del cliente, y la empresa prevé incorporar nuevas variantes (motocicletas, buses o vehículos eléctricos). Si la lógica principal instanciara directamente `new Auto(...)`, `new Camioneta(...)` o `new Camion(...)`, quedaría acoplada a las clases concretas y cada nuevo tipo obligaría a modificar el código existente, en contra del principio abierto/cerrado (Meyer, 1997; Martin, 2017).

Requisitos identificados

Requisitos funcionales:

- Registrar vehículos de tipo Auto, Camioneta y Camión.
- Almacenar placa, marca, modelo, año de fabricación y avalúo de cada vehículo.
- Ofrecer una operación común para calcular el valor de la matrícula, con una lógica específica por tipo.
- Seleccionar el tipo de vehículo durante la ejecución, a partir de la información del cliente.

Requisitos de diseño:

- Evitar que la lógica principal dependa de las clases concretas Auto, Camioneta y Camión.
- Centralizar las instrucciones de creación en componentes especializados, en lugar de distribuirlas por la aplicación.
- Facilitar la incorporación de nuevos tipos de vehículos sin modificar el código cliente.
- Tratar a todos los vehículos de forma polimórfica mediante un contrato común.

Selección y justificación del patrón

Se selecciona el patrón **Factory Method**, cuyo propósito es "definir una interfaz para crear un objeto, pero dejar que sean las subclases quienes decidan qué clase instanciar" (Gamma et al., 1994). En este escenario, la clase creadora `CreadorVehiculo` declara el método de fabricación `crearVehiculo(...)` y cada creador concreto decide qué vehículo instanciar. De este modo, una vez elegido el creador, el resto del proceso trabaja exclusivamente con las abstracciones `CreadorVehiculo` y `Vehiculo`, lo que satisface los requisitos de desacoplamiento y extensibilidad (Freeman y Robson, 2020; Ortiz Ochoa, 2026). Asimismo, asignar la responsabilidad de creación a clases especializadas es coherente con los principios GRASP de *Creador*, *Alta cohesión* y *Variaciones protegidas* (Larman, 2004).

Durante el análisis se contrastaron dos alternativas. Una *Simple Factory* con una estructura `switch` resolvería la creación en un único punto, pero obligaría a modificar esa clase cada vez que se agregue un tipo de vehículo; además, no figura en el catálogo de Gamma et al. (1994) como patrón, sino como una práctica de programación (Freeman y Robson, 2020). Por otra parte, *Abstract Factory* está orientado a crear **familias** de productos relacionados, situación que no se presenta en el enunciado, donde existe una sola jerarquía de productos (Shvets, s.f.-b). Por ello, Factory Method es la alternativa con menor complejidad que cumple todos los requisitos.

Ventajas	Limitaciones
Elimina la creación directa de vehículos concretos en la lógica principal.	Incrementa el número de clases: un creador por cada tipo de vehículo.
Permite tratar a todos los vehículos mediante la clase abstracta `Vehiculo`.	El diseño inicial resulta más extenso que una instanciación directa.
Facilita incorporar nuevos tipos sin modificar el código cliente (principio abierto/cerrado).	Persiste un punto de configuración donde se elige el creador concreto.
Concentra la responsabilidad de creación en componentes especializados (responsabilidad única).	Los datos propios de cada tipo deben llegar al creador concreto por otra vía (constructor).

Participantes del patrón

Participante	Clase	Responsabilidad
Product	`Vehiculo` (abstracta)	Define los atributos comunes y el contrato `calcularMatricula()`.
ConcreteProduct	`Auto`, `Camioneta`, `Camion`	Implementan su propia regla de cálculo de matrícula.
Creator	`CreadorVehiculo` (abstracta)	Declara el método de fabricación `crearVehiculo(...)`, que retorna un `Vehiculo`.
ConcreteCreator	`CreadorAuto`, `CreadorCamioneta`, `CreadorCamion`	Sobrescriben `crearVehiculo(...)` e instancian el producto concreto.
Cliente	`Main`	Selecciona el creador en tiempo de ejecución y trabaja solo con las abstracciones.

Diagrama de clases

El diagrama se elaboró en dos iteraciones. La primera versión (Figura 1) se obtuvo a partir del análisis del grupo y de la revisión con IA. La versión final (Figura 2) incorpora la firma de `crearVehiculo(...)` con los atributos comunes y los constructores de los creadores concretos que reciben los datos propios de cada tipo. Se utiliza generalización para las jerarquías de productos y creadores, y dependencias estereotipadas `<<create>>` desde cada creador concreto hacia el producto que instancia, conforme a la notación de UML 2.5.1 (Object Management Group, 2017; Fowler, 2003).

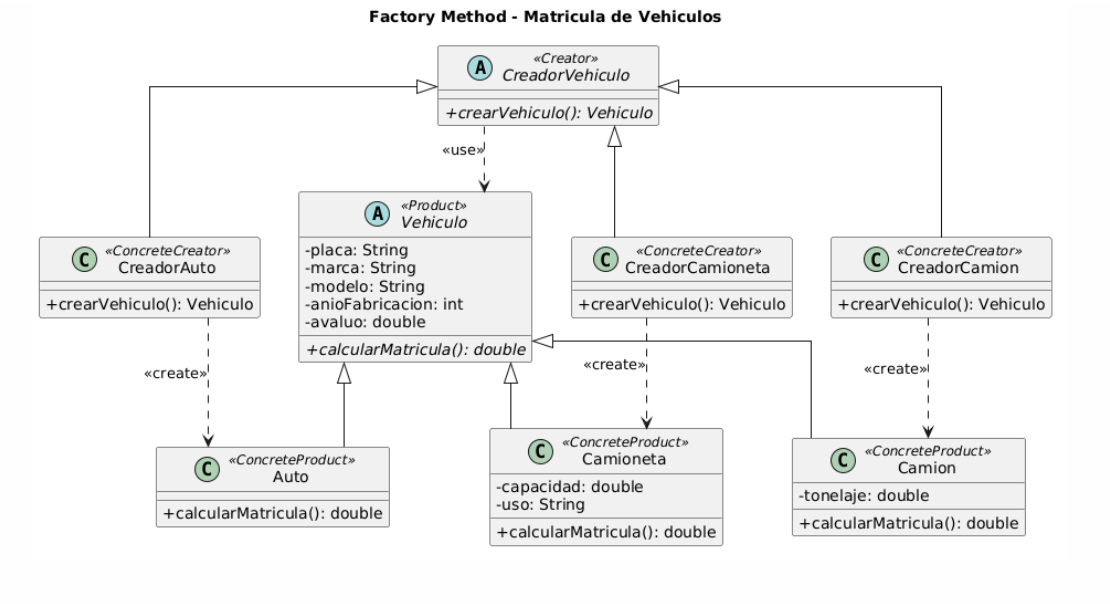


Figura 1. Diagrama de clases inicial de Factory Method.

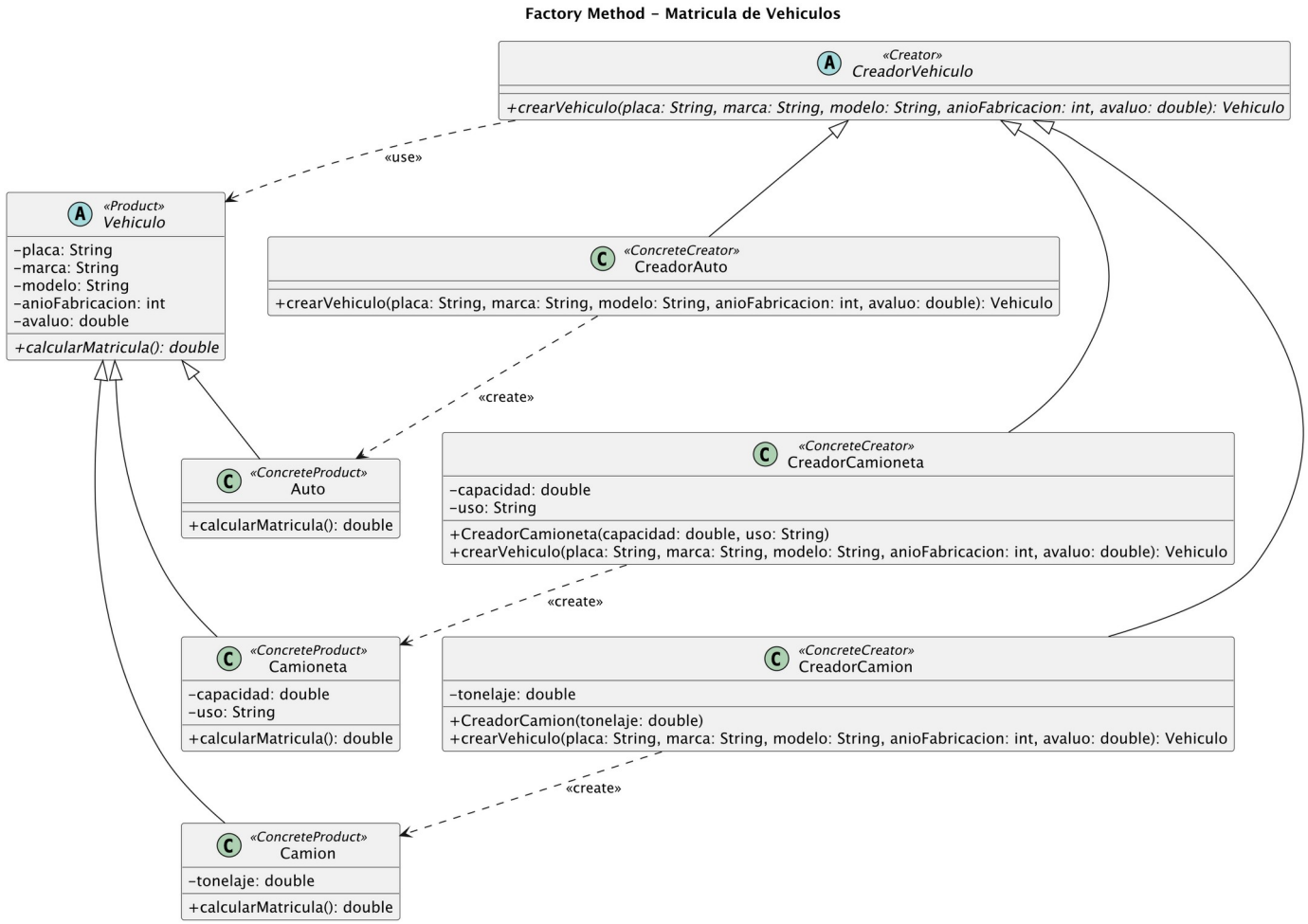


Figura 2. Diagrama de clases final de Factory Method (fuente: `diagrama.tex`, PlantUML).

Código fuente

- **Repositorio:** https://github.com/miguesj-hub/patrones-diseno-actividad-01
- **Ubicación:** carpeta `codigo-escenario-1/`, paquete `vehiculos` (Java 17).
- **Ejecución:** `javac -d out $(find src -name "*.java")` y luego `java -cp out vehiculos.Main`.

La correspondencia entre el diagrama UML y el código es directa:

Participante	Clase en el código
Product	`vehiculos.producto.Vehiculo`
ConcreteProduct	`vehiculos.producto.Auto`, `Camioneta`, `Camion`
Creator	`vehiculos.creador.CreadorVehiculo`
ConcreteCreator	`vehiculos.creador.CreadorAuto`, `CreadorCamioneta`, `CreadorCamion`
Cliente	`vehiculos.Main`

El siguiente extracto muestra el método de fabricación y el punto del cliente que trabaja únicamente con las abstracciones:

```
public abstract class CreadorVehiculo {
    public abstract Vehiculo crearVehiculo(String placa, String marca, String modelo,
                                           int anioFabricacion, double avaluo);
}

public class CreadorCamion extends CreadorVehiculo {
    private final double tonelaje;
    public CreadorCamion(double tonelaje) { this.tonelaje = tonelaje; }
    @Override
    public Vehiculo crearVehiculo(String placa, String marca, String modelo,
                                  int anioFabricacion, double avaluo) {
        return new Camion(placa, marca, modelo, anioFabricacion, avaluo, tonelaje);
    }
}

// Main.registrar(): no depende de Auto, Camioneta ni Camion
Vehiculo vehiculo = creador.crearVehiculo(placa, marca, modelo, anio, avaluo);
System.out.printf("Valor de matrícula: $%.2f%n", vehiculo.calcularMatricula());
```

Las reglas de cálculo no están definidas en el enunciado; por ello, el grupo asumió fórmulas con fines académicos: **Auto** = 2 % del avalúo + 1 % por año de antigüedad (máximo 10 %); **Camioneta** = 3 % del avalúo + 20 por tonelada de capacidad, con un recargo de 15 % si el uso es comercial; **Camión** = 4 % del avalúo + 50 por tonelada. A continuación se muestra una ejecución de prueba registrando una camioneta de uso comercial:

```
=== Registro de vehículos y cálculo de matrícula ===
Tipo de vehículo: 1) Auto  2) Camioneta  3) Camión
Opción: 2   Capacidad (t): 1.5   Uso: comercial
Placa: ABC-123   Marca: Toyota   Modelo: Hilux   Año: 2020   Avalúo: 30000


Vehículo registrado: Camioneta [placa=ABC-123, marca=Toyota, modelo=Hilux,
  año=2020, avalúo=30000,00] capacidad=1,50 t, uso=comercial
Valor de matrícula: $1069,50
```

Para incorporar un nuevo tipo, por ejemplo `Motocicleta`, basta con crear la subclase `Motocicleta extends Vehiculo` y su creador `CreadorMotocicleta extends CreadorVehiculo`, y registrar la opción en `Main.seleccionarCreador()`; el método `registrar()` y las clases existentes no se modifican.

Ajustes realizados durante la implementación:

Cambio	Justificación
Métodos protegidos `validarNoNegativo()` y `antiguedad()` en `Vehiculo`.	Evitan duplicar validaciones y el cálculo de antigüedad en cada subclase, sin alterar el contrato público del diagrama.
Validaciones en los constructores (placa obligatoria, año entre 1900 y el actual, valores no negativos).	Impiden crear vehículos inconsistentes sobre los que no se podría calcular la matrícula.
Atributos declarados `final`.	Un vehículo no debe cambiar sus datos una vez registrado.
Datos propios (`capacidad`, `uso`, `tonelaje`) recibidos por el constructor del creador concreto.	Conserva una firma común de `crearVehiculo(...)` para todos los creadores, condición necesaria para el polimorfismo.
Cliente `Main` con menú de selección.	Demuestra que, tras elegir el creador, el proceso solo usa `CreadorVehiculo` y `Vehiculo`.

Uso crítico de inteligencia artificial

- **Prompt principal:**

  Se utilizó ChatGPT como asistente de revisión del análisis previo del grupo y de la propuesta de diagrama. Conversación completa: enlace a la conversación.

> *Actúa como un asistente especializado en patrones de diseño de software y modelado UML. Estoy desarrollando una actividad académica sobre patrones creacionales. Para el primer escenario he seleccionado el patrón Factory Method y ya realicé un análisis previo del problema.*
> *Objetivo: necesito que revises mi análisis y me ayudes a elaborar una propuesta de diagrama de clases UML utilizando Factory Method.*
> *Instrucciones: (1) revisa el enunciado original y compáralo con mi análisis para identificar errores, requisitos faltantes o aspectos por aclarar; (2) evalúa si la selección de Factory Method es adecuada e indica brevemente por qué; (3) propón las clases necesarias según la estructura tradicional: Product, ConcreteProduct, Creator y ConcreteCreator; (4) identifica los atributos y métodos principales de cada clase, considerando únicamente los requisitos del escenario; (5) explica las relaciones entre las clases y el tipo de relación UML que corresponde; (6) genera una propuesta de diagrama en PlantUML; (7) justifica brevemente las principales decisiones de diseño; (8) identifica limitaciones o aspectos por mejorar.*
> *Consideraciones: utiliza un lenguaje sencillo; no agregues clases o funcionalidades innecesarias; respeta la estructura de Factory Method del material académico; no inventes requisitos y, si asumes algo, indícalo; no generes código todavía; considera el diagrama como una propuesta que debe revisarse antes de implementarse.*
> *[Se adjuntaron el enunciado original y el análisis previo del grupo.]*

- **Recomendación aceptada (con justificación):** definir `Vehiculo` como **clase abstracta** en lugar de interfaz.

  Se acepta porque todos los vehículos comparten estado (placa, marca, modelo, año de fabricación y avalúo) y comportamiento de validación, mientras que `calcularMatricula()` debe ser específico de cada tipo. Una clase abstracta permite reutilizar ese estado común y, a la vez, obligar a cada subclase a implementar su cálculo. La recomendación coincidía, además, con el análisis inicial del grupo, lo que reforzó la decisión.

- **Recomendación modificada (con justificación):** la firma del método de fabricación `crearVehiculo()`.

  La IA propuso un método de fabricación sin parámetros. El grupo lo modificó para que reciba los atributos comunes (`placa`, `marca`, `modelo`, `anioFabricacion`, `avaluo`), ya que sin ellos no es posible construir un vehículo válido ni registrar la información que exige el enunciado. Los atributos propios de cada tipo se trasladaron al constructor del creador concreto, de modo que todos los creadores conservan la misma firma y el cliente puede invocarlos de manera polimórfica. La modificación no altera la estructura del patrón y responde a una necesidad concreta del escenario.

Escenario 02: Registro de activos fijos (Builder)

Justificación

Identificación del problema de creación de objetos

La institución necesita registrar activos fijos de naturaleza diversa (computadores, monitores, impresoras, muebles, vehículos y otros bienes). Todo activo exige tres datos obligatorios —código institucional, nombre o descripción y precio de adquisición— y admite hasta diez datos opcionales cuya presencia depende del tipo de bien: un computador requiere marca, modelo, número de serie, procesador y garantía, mientras que una mesa solo necesita material, dimensiones, ubicación y custodio.

El problema de creación surge por la **combinación de atributos obligatorios y opcionales**. Resolverlo con constructores sobrecargados conduce al antipatrón del *constructor telescópico*, en el que llamadas como `new ActivoFijo("AF001", "Computador", 1200, "Dell", "Latitude", null, ...)` resultan difíciles de leer, propensas a errores de orden de parámetros y obligan a pasar valores `null` para datos que no aplican. La alternativa de constructor vacío con métodos *set* tampoco es adecuada, porque permite que el objeto exista en un estado inconsistente durante su configuración (Bloch, 2018).

Requisitos identificados

Requisitos funcionales:

- Registrar distintos tipos de activos institucionales.
- Exigir código institucional, nombre o descripción y precio de adquisición.
- Registrar únicamente los atributos opcionales que correspondan a cada activo, incluidas características técnicas variables.

Requisitos de diseño:

- Construir los objetos de forma progresiva, haciendo explícito qué dato se incorpora en cada paso.
- Evitar constructores sobrecargados y valores vacíos o `null` para atributos que no aplican.
- Entregar al final un objeto correctamente configurado, validando los datos obligatorios.
- Separar el proceso de construcción de la representación final del activo.

Selección y justificación del patrón

Se selecciona el patrón **Builder**, cuyo propósito es "separar la construcción de un objeto complejo de su representación, de modo que el mismo proceso de construcción pueda crear diferentes representaciones" (Gamma et al., 1994). Bloch (2018) lo recomienda expresamente cuando una clase presenta muchos parámetros, en particular si varios de ellos son opcionales, porque combina la seguridad del constructor telescópico con la legibilidad de los métodos *set*. En este escenario, `ConstructorActivoFijo` acumula los datos paso a paso y solo entrega el `ActivoFijo` en `obtenerActivo()`, tras validar los obligatorios, mientras que `DirectorActivoFijo` encapsula la secuencia común de construcción de la base obligatoria (Ortiz Ochoa, 2026; Shvets, s.f.-a).

Se descartó *Factory Method* porque el problema no radica en decidir **qué clase** instanciar, sino en **cómo** configurar un único tipo de objeto con numerosas combinaciones de atributos. Asimismo, se descartó una jerarquía de productos por tipo de activo, ya que el enunciado no asigna comportamiento diferente a cada bien, sino solo datos distintos.

Ventajas	Limitaciones
Construcción paso a paso, con métodos que hacen explícito cada dato incorporado.	Requiere clases adicionales (Builder, ConcreteBuilder y Director).
Elimina los constructores telescópicos y los valores `null` en los parámetros.	Duplica la declaración de atributos entre el builder y el producto.
Permite validar los datos obligatorios antes de entregar el objeto.	La obligatoriedad se verifica en tiempo de ejecución, no en compilación.
Admite configuraciones distintas con el mismo proceso de construcción.	Puede ser excesivo para objetos con pocos atributos.
Favorece productos inmutables y consistentes.	Exige definir con claridad la responsabilidad del Director frente al cliente.

Participantes del patrón

Participante	Clase	Responsabilidad
Product	`ActivoFijo`	Representa el activo final; es inmutable y solo puede crearlo el builder.
Builder	`ActivoFijoBuilder` (interfaz)	Declara los pasos de construcción y la operación `obtenerActivo()`.
ConcreteBuilder	`ConstructorActivoFijo`	Mantiene los datos en construcción, valida los obligatorios y crea el producto.
Director	`DirectorActivoFijo`	Inicia la construcción y establece la base obligatoria (`construirBase`).
Cliente	`Main`	Usa el Director para la base y agrega en el builder los opcionales de cada activo.

Diagrama de clases

La Figura 3 corresponde a la propuesta inicial y la Figura 4 al diagrama final, ajustado durante la implementación. Los cambios principales fueron: la relación entre `ConstructorActivoFijo` y `ActivoFijo` pasó de asociación a **dependencia** `<<create>>`, porque el builder no conserva el producto como atributo; se listaron los campos temporales del ConcreteBuilder; se agregaron multiplicidades; y se emplearon los tipos `BigDecimal` y `LocalDate` para valores monetarios y fechas.

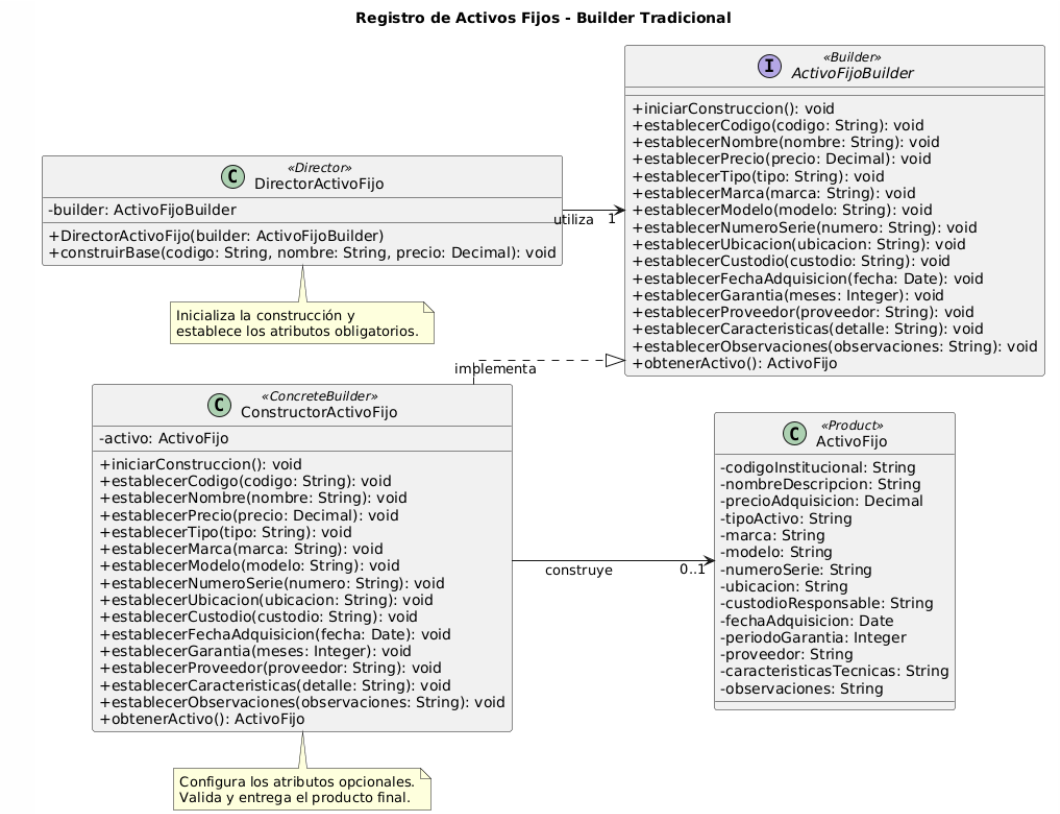


Figura 3. Diagrama de clases inicial de Builder.



Figura 4. Diagrama de clases final de Builder (fuente: `diagrama.tex`, PlantUML).

Código fuente

- **Repositorio:** https://github.com/miguesj-hub/patrones-diseno-actividad-01
- **Ubicación:** carpeta `codigo-escenario-2/`, paquete `activos` (Java 17).
- **Ejecución:** `javac -d out $(find src -name "*.java")` y luego `java -cp out activos.Main`.

La correspondencia entre el diagrama UML y el código es la siguiente:

Participante	Clase en el código
Product	`activos.construccion.ActivoFijo`
Builder	`activos.construccion.ActivoFijoBuilder`
ConcreteBuilder	`activos.construccion.ConstructorActivoFijo`
Director	`activos.construccion.DirectorActivoFijo`
Cliente	`activos.Main`

El extracto muestra la construcción progresiva de un computador: el Director fija los obligatorios y el cliente agrega solo los opcionales que corresponden; `obtenerActivo()` valida y entrega el producto:

```
director.construirBase("AF001", "Computador portátil", new BigDecimal("1200.00"));
builder.establecerTipo("Computador");
builder.establecerMarca("Dell");
builder.establecerModelo("Latitude 5440");
builder.establecerNumeroSerie("DL-5440-88213");
builder.establecerCaracteristica("Procesador", "Intel Core i7");
builder.establecerGarantia(36);
ActivoFijo computador = builder.obtenerActivo();


// ConstructorActivoFijo.obtenerActivo()
if (esVacio(codigo)) throw new IllegalStateException("Falta el código institucional");
if (esVacio(nombre)) throw new IllegalStateException("Falta el nombre o descripción");
if (precio == null || precio.signum() <= 0)
    throw new IllegalStateException("Falta el precio de adquisición o no es mayor que cero");
ActivoFijo resultado = new ActivoFijo(codigo, nombre, precio, tipo, marca, ...);
iniciarConstruccion();   // reinicia el builder para el siguiente activo
return resultado;
```

Resultado de la ejecución: dos activos con configuraciones distintas y un tercero rechazado por carecer de datos obligatorios.

```
=== Registro de activos fijos ===
ActivoFijo
  código: AF001 | nombre: Computador portátil | precio: 1200.00 | tipo: Computador
  marca: Dell | modelo: Latitude 5440 | número de serie: DL-5440-88213
  fecha de adquisición: 2026-03-15 | garantía (meses): 36 | proveedor: Tecnología Andina S.A.
  características: {Procesador=Intel Core i7, Memoria RAM=16 GB, Almacenamiento=512 GB SSD}
ActivoFijo
  código: AF002 | nombre: Mesa de reuniones | precio: 350.00 | tipo: Mueble
  ubicación: Cuenca - Sala de juntas 2 | custodio: Departamento Administrativo
  características: {Material=Madera de roble, Dimensiones=2,40 x 1,10 m}
Activo rechazado: Falta el nombre o descripción
```

Ajustes realizados durante la implementación:

Cambio	Justificación
`ActivoFijo` inmutable: campos `final`, constructor de paquete, sin *setters* y mapa de características no modificable.	Garantiza que un activo construido y validado no pueda quedar en un estado inválido.
El ConcreteBuilder guarda los datos en campos temporales y crea el producto en `obtenerActivo()`.	Consecuencia de la inmutabilidad; por ello la relación en UML es una dependencia `<<create>>`.
Validación de obligatorios en `obtenerActivo()` con `IllegalStateException`.	Cumple el requisito de entregar un objeto “correctamente configurado”.
Reinicio del builder al finalizar y `iniciarConstruccion()` en el Director.	Permite reutilizar el builder sin arrastrar datos del activo anterior.
`establecerGarantia` rechaza valores negativos y `establecerCaracteristica` rechaza claves vacías.	Evita datos opcionales incoherentes.
Métodos del builder con retorno `void` (estilo clásico, no encadenado).	Mantiene la estructura de Gamma et al. (1994) con Director; la variante fluida de Bloch (2018) volvería innecesario al Director.

Uso crítico de inteligencia artificial

- **Prompt principal:**

  Conversación completa: enlace a la conversación.

> *Actúa como un asistente especializado en patrones de diseño de software y modelado UML. Estoy desarrollando una actividad académica sobre patrones creacionales. Para el segundo escenario he seleccionado el patrón Builder y ya realicé un análisis previo del problema.*
> *Objetivo: necesito que revises mi análisis y me ayudes a elaborar una propuesta de diagrama de clases UML utilizando el patrón Builder en su estructura tradicional.*
> *Instrucciones: (1) revisa el enunciado y compáralo con mi análisis; (2) evalúa si Builder es adecuado; (3) propón las clases según la estructura tradicional: Product, Builder, ConcreteBuilder y Director; (4) identifica atributos y métodos principales; (5) explica cómo se manejarán los atributos obligatorios y opcionales; (6) explica las relaciones UML entre las clases; (7) genera una propuesta en PlantUML; (8) justifica las decisiones de diseño; (9) identifica limitaciones o mejoras.*
> *Consideraciones: utiliza un lenguaje sencillo; no agregues clases innecesarias; respeta la estructura tradicional de Builder del material académico; considera inicialmente ActivoFijo (Product), ActivoFijoBuilder (Builder), ConstructorActivoFijo (ConcreteBuilder) y DirectorActivoFijo (Director), pero evalúa si sus responsabilidades son adecuadas; no crees una clase por cada tipo de activo salvo que el enunciado lo justifique; no inventes requisitos; no generes código todavía.*
> *[Se adjuntaron el enunciado original y el análisis previo del grupo.]*

- **Recomendación aceptada (con justificación):** utilizar una **única clase producto** `ActivoFijo`, sin jerarquía por tipo de activo.

  Se acepta porque los computadores, impresoras, muebles o vehículos no presentan comportamiento diferente en el enunciado; solo difieren en los datos que registran. Crear una subclase por tipo habría incrementado la complejidad sin aportar valor y habría acercado el diseño a un problema de selección de clases, propio de Factory Method. Con una sola clase, el Builder permite configurar cada activo únicamente con los atributos que le corresponden.

- **Recomendación modificada (con justificación):** el tipo del atributo `caracteristicasTecnicas`.

  La IA propuso modelar las características técnicas como un `String`. El grupo lo modificó a `Map<String, String>`, cargado mediante `establecerCaracteristica(clave, valor)`, porque el enunciado menciona datos heterogéneos por tipo de activo (procesador, material, dimensiones) que no conviene concatenar en un texto libre ni convertir en atributos fijos de la clase. El mapa ofrece una estructura organizada y extensible, y se expone como colección no modificable para preservar la inmutabilidad del producto.

Adicionalmente, el grupo **rechazó** mantener la asociación `ConstructorActivoFijo → ActivoFijo` sugerida en la propuesta inicial, y la reemplazó por una dependencia `<<create>>`, ya que el ConcreteBuilder no conserva al producto como atributo, sino que lo crea al finalizar la construcción (Object Management Group, 2017).

Conclusiones de la actividad

El desarrollo de la actividad permitió constatar que la selección de un patrón creacional parte de una lectura cuidadosa del enunciado, más que de la memorización del catálogo. En el Escenario 01, la necesidad de decidir en tiempo de ejecución el tipo de vehículo y de incorporar variantes futuras sin alterar la lógica principal condujo a Factory Method: el cliente opera únicamente con `CreadorVehiculo` y `Vehiculo`, de modo que agregar una motocicleta solo exige una nueva subclase de producto y su creador, en coherencia con el principio abierto/cerrado. En el Escenario 02, la combinación de atributos obligatorios y opcionales justificó el uso de Builder, que sustituye los constructores telescópicos por una construcción paso a paso, legible y validada antes de entregar el objeto.

Respecto al uso de inteligencia artificial, el asistente resultó útil para contrastar el análisis inicial y obtener una primera propuesta de diagrama; sin embargo, sus sugerencias debieron evaluarse frente al enunciado y al material de la asignatura. Se aceptaron decisiones sustentadas, como la clase abstracta `Vehiculo` o el producto único `ActivoFijo`, y se modificaron otras, como la firma de `crearVehiculo()` y el tipo de las características técnicas. Finalmente, la implementación obligó a refinar el UML para mantener la correspondencia con el código, lo que evidencia que diseño e implementación deben revisarse de forma iterativa y que la IA complementa, pero no reemplaza, el criterio del diseñador.

Referencias bibliográficas

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Freeman, E., y Robson, E. (2020). *Head First design patterns: Building extensible and maintainable object-oriented software* (2.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Meyer, B. (1997). *Object-oriented software construction* (2.ª ed.). Prentice Hall.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), versión 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Ortiz Ochoa, M. (2026). *Unidad 01: Patrones creacionales* [Material de clase, secciones 01, 02 y 03]. Maestría en Software, Universidad Politécnica Salesiana.

Shvets, A. (s.f.-a). *Builder*. Refactoring.Guru. https://refactoring.guru/es/design-patterns/builder

Shvets, A. (s.f.-b). *Factory Method*. Refactoring.Guru. https://refactoring.guru/es/design-patterns/factory-method